P2212R0
Relax Requirements for time_point::clock

Published Proposal, 2020-08-11

This version:
https://wg21.link/P2212R0

Latest published version:
https://wg21.link/P2212

Authors:
Alexey Dmitriev
Howard Hinnant

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

1 Abstract

2 Revision History

2.1 Revision 0

3 Motivation

4 Wording

5 Effect on existing code

6 Effect on implementations

7 Acknowledgement

§ 1. Abstract

We propose to relax the requirements on the `Clock` template parameter of `std::chrono::time_point`.

§ 2. Revision History

§ 2.1. Revision 0

Initial revision.

§ 3. Motivation

It is sometimes useful to give `time_point` a `Clock` template argument that does not meet the *Cpp17Clock* requirements. The most obvious use case is the `local_t` introduced in C++20. This "clock" is not really a clock at all. It has no functionality for getting the current time. Nevertheless, C++20 utilizes `time_point<local_t, Duration>` to represent a family of time points that do not represent instants in time until they are paired with a specific `time_zone`. This gives "local time" a distinct type from "system/UTC time" in order to reduce the possibility of the programmer confusing these two types in their code.

Since the introduction of `local_t`, more use cases for "not-quite-clocks" have become apparent:

- Some people need a stateful clock. This requires a non-static `now()` function.
- Some people would like to represent "time of day" as a distinct `time_point`.
- It is sometimes useful to represent time points as defined by some external system not controlled by you, where you can’t generate the current time_point easily or at all.

  One example might be value of timestamp counter on different computer. You can’t easily generate the current value, but you can still meaningfully manipulate existing values generated by other means: compare them, calculate differences between them, etc.

§ 4. Wording

Change the clause 1 in [**time.point**]:

Clock shall ~~either meet the *Cpp17Clock* requirements ([**time.clock.req**]) or be the type `local_t`~~ have the nested type `duration` if the instantiation defaults the template parameter `Duration`.

§ 5. Effect on existing code

This change should not affect any conforming code because the requirement is only relaxed.

§ 6. Effect on implementations

This change should not have an effect on most of implementations because in practice they do not use any of the members of `Clock` outside of **[thread]**. **[thread.req.paramname]** places additional requirements on the `Clock` template parameter, namely that `is_clock_v<Clock>` is `true`. This change is effectively already proven to be safe because of the existence of `std::chrono::time_point<std::chrono::local_t, Duration>`, where `std::chrono::local_t` is an empty class.

§ 7. Acknowledgement

Thanks to all those who provided the motivating use cases for this change.